

Ultimate 9-Point Website Security Checklist

A website security checklist serves as a crucial resource for security professionals, ethical hackers, and DevSecOps teams in maintaining the security of their web applications. This comprehensive guide outlines best practices and essential steps to protect websites from data breaches, hacking attempts, malware infections, and other vulnerabilities.

In the current digital environment, where cyberattacks are growing in number and sophistication, implementing a robust website security checklist can aid organizations in protecting their valuable assets and maintaining user trust. By following this list, you can effectively reduce potential risks associated with your online presence.

Why You Need a Website Security Checklist

Here are three reasons a website security checklist can be significant for your organization:

- 1. Risk mitigation:** A well-designed checklist helps pinpoint potential weaknesses in your web application infrastructure before they escalate into critical issues or become targets for malicious actors.
- 2. Compliance maintenance:** Numerous industries have specific regulatory requirements concerning data protection and privacy. By adopting proper website security measures, you ensure compliance with relevant laws and regulations.
- 3. Protecting user trust:** Guaranteeing the security of your users' sensitive information is crucial for fostering trust between them and your organization. A comprehensive website security checklist offers peace of mind for both parties involved.

In this article, we cover the following website security checklist steps:

- [1. Implement Sitewide SSL](#)
- [2. Validate the SSL Certificate](#)
- [3. Implement SHA256 Encryption for Passwords](#)
- [4. Activate HTTP Strict Transport Security \(HSTS\)](#)

- [5. Employ Secure Cookies](#)
- [6. Harden Web Server Processes](#)
- [7. Guarantee Input Validation in Forms](#)
- [8. Defend Against Denial of Service](#)
- [9. Regularly Assess Configurations](#)

1. Implement Sitewide SSL

SSL certificates protect data from eavesdropping or tampering and improve search engine rankings. There are several certificate types available:

- **Domain Validation (DV):** Basic encryption, issued after verifying domain ownership.
- **Organization Validation (OV):** Additional validation steps to ensure organizational legitimacy.
- **Extended Validation (EV):** Rigorous vetting processes for maximum security.

In order to implement an SSL certificate, you will need to transition your website from HTTP to HTTPS. This involves several steps:

1. Obtain an SSL credential from a reliable Certificate Authority (CA).
2. Create a private key pair on your web server.
3. Install the SSL certificate on your web server.
4. Update all internal and external links to use HTTPS.
5. Set up proper redirects from HTTP to HTTPS.

2. Validate the SSL Certificate

Take the following steps to ensure your SSL certificate is valid and secure:

- **Verify certificate expiration date:** Renew your certificate before it expires to avoid disruptions. Use online tools to check its validity period.
- **Ensure Adequate Domain Coverage:** There are three types of certificates in terms of coverage of website domains. Single Domain Certificates cover one domain name, Wildcard Certificates cover all subdomains under a single domain, and Multidomain Certificates cover multiple domains in a single cert.
- **Confirm robust encryption algorithms:** Verify that your SSL certificate supports up-to-date algorithms like Elliptic Curve Cryptography (ECC) and RSA-2048.
- **Choose a trustworthy certificate authority:** Select a trusted CA considering factors such as price, support, issuance speed, and compatibility.

3. Implement SHA256 Encryption for Passwords

SHA256 is a cryptographic hash function that generates a fixed-size output from any input data. It is commonly used to encrypt passwords stored by websites, to prevent their exposure if the server is compromised by attackers.

SHA256 encryption provides a higher level of security compared to older algorithms like MD5 and SHA1. There are several reasons you should prefer SHA256 over other algorithms:

- **Enhanced security:** Improved resistance against attacks.
- **Faster performance:** Efficient processing without compromising performance.
- **Widespread adoption:** Compatible with various systems while maintaining high-security standards.

Here are a few tips for Implementing secure password storage

- **Generate unique salts:** Create random values to combine with the user's password before hashing.
- **Use a slow hash function:** Consider algorithms like bcrypt or Argon2 for secure password storage.
- **Regularly update your hashing algorithm:** Keep up-to-date with security best practices.

4. Activate HTTP Strict Transport Security (HSTS)

HTTP Strict Transport Security (HSTS) enforces communication over HTTPS, preventing downgrade attacks and cookie hijacking.

- Implement HSTS by adding a Strict-Transport-Security header to your pages with a max-age value.
- Consider submitting your domain for [HSTS preloading](#) to be included in an official list of HSTS-supporting websites.

5. Employ Secure Cookies

[Secure cookies](#) have the Secure attribute set, meaning they are only transmitted via HTTPS connections. To implement secure cookies:

- Set the Secure attribute
- Adding an HttpOnly flag
- Establishing a [SameSite policy](#)
- Regularly audit your cookies to ensure they are securely configured

You can configure secure cookies on all popular web servers, for example:

- **On Apache HTTP Server**, use the `mod_headers` Module
- **On NGINX**, use the `ngx_http_headers_module`

On Microsoft IIS, use the System.WebServer Configuration Section (HTTPCookies)

6. Harden Web Server Processes

Follow these best practices to minimize security risks:

- Select a trustworthy hosting provider with robust security features.
- Keep software up-to-date to avoid vulnerabilities.
- Configure proper permissions and access controls for user accounts, passwords, data encryption, and file permissions.
- Implement regular monitoring and scanning using tools like malware scanners.
- Deploy a Web Application Firewall (WAF).

Securing web server processes and ensuring form input validation are crucial steps for website security, protecting against threats like cross-site scripting and SQL injection.

7. Guarantee Input Validation in Forms

Client-side and Server-side Validation

Input validation should be executed on both client-side and server-side for optimal security. Here is how to do it:

- **Client-side validation:** Use JavaScript libraries like jQuery Validation Plugin or HTML5 form attributes to enforce input restrictions in the browser.
- **Server-side validation:** Apply strong server-level checks using programming languages like PHP, Python, or Ruby on Rails to confirm submitted data.

Data Sanitization Methods

Sanitize user-submitted data before processing it within your application. This can protect against many types of attacks, in particular SQL injection.

In general, website developers should follow secure coding guidelines. Some common techniques for protecting against unsafe inputs include:

- **Escaping:** Use suitable escaping functions for different contexts, such as HTML entities encoding or parameterized queries.
- **Allowlisting:** Establish a list of permitted characters and input patterns.
- **Deploy a Web Application Firewalls:** A Web Application Firewall (WAF) can offer an extra layer of protection by blocking malicious requests.
- **Regularly update database software:** Keep your database software up-to-date with the most recent security patches and updates to protect against potential exploits.

8. Defend Against Denial of Service

To protect your website from DoS attacks, follow these best practices:

- **Implement rate limiting:** Restrict the number of requests a single user can make within a specified time frame to prevent automated bots from overloading your server resources.
- **Use Content Delivery Networks (CDNs):** CDNs distribute your website's content across multiple servers worldwide, reducing latency and improving load times while also helping mitigate DoS attacks.
- **Deploy Web Application Firewalls (WAFs):** WAFs monitor incoming web traffic and filter out potentially harmful requests based on predefined rulesets or custom configurations.
- **Monitor and analyze traffic patterns:** Regularly monitor your website traffic for unusual patterns or spikes in activity to detect potential DoS attacks early.
- **Implement load balancing solutions:** Load balancing solutions distribute incoming traffic across multiple servers to ensure that no single server is overwhelmed during high-traffic periods or targeted by a DoS attack.

9. Regularly Assess Configurations

Conduct regular tests on your website's configurations to maintain a secure environment and stay ahead of potential threats:

- **Review your security settings:** Periodically evaluate your security settings, including server configurations, firewall rulesets, access control policies, and authentication mechanisms.
- **Automate vulnerability scanning:** Perform regular vulnerability scanning using automated tools. Shift vulnerability tests left, conducting them as early as possible in the development process.

Penetration testing: Perform penetration testing, also known as ethical hacking, to simulate real-world attacks on your website, identify vulnerabilities, and assess the effectiveness of your security measures.

Company

Leadership

Careers

Partners

Newsroom

Contact Us

Knowledge

Center

Application Security

Penetration Testing

Cloud Security

Hacking

Cybersecurity

Attacks

DevSecOps

Resources

Blog

Documentation

Leaderboard

Partner Portal

Resources

Contacted
by a
hacker?

